Converting charisma to the GPU — Methodology

charisma-gpu: R package charisma 1.0.0 (Schwartz et al.) and recolorize 0.2.0 (Weller) re-implemented in PyTorch | with a CLUT for *Rudbeckia hirta* ray, bullseye and disk colour

This document records how the R pipeline was read, what the Python version does at each step and why, how the two were checked against each other, and how the colour look-up table (CLUT) for *Rudbeckia hirta* was built. Numbers marked **measured** come from runs on an RTX 4060 laptop GPU.

1. Summary

	Result
Agreement with R	40/40 real flower cut-outs: same colour classes and k, every proportion within 0.0005. 16/16 synthetic images, with each of two CLUTs, within 0.002 (automated tests). **measured**
Speed	R: 2.4 s per image, in serial (pavo = FALSE). charisma-gpu: all 126,653 *R. hirta* segments in ~2 min, 1,057 segments/s, limited by PNG reading. **measured**
Not reproduced	pavo adjacency statistics (pavo = TRUE), interactive sessions, diagnostic plots
Added	Per-pixel CLUT classification; an *R. hirta* CLUT separating bullseye, disk and ray; and a spatial step that counts bullseye only on the petals, not on red disks (section 7)

2. What R charisma does

The source of charisma 1.0.0 and recolorize 0.2.0 was read function by function from the installed packages. With the defaults (`bins = 4, cutoff = 20, threshold = 0`), `charisma(img)` runs:

Step	R function	What it does
Read	charisma_readImage	imager::load.image; values in [0, 1]
Background	recolorize::backgroundIndex	a pixel is background when round(alpha) < 1, i.e. alpha < 128 of 255
Bin	recolorize:::colorClustersHist	sRGB cut into 4 equal right-closed intervals per channel (64 bins); a bin's colour is the mean of its pixels; bins in expand.grid order
Merge	charisma_recluster, charisma_hclust_color	empty bins dropped; bin colours to CIE Lab (D65); Euclidean distance; hclust(complete); cutree(h = 20); each group's colour = size-weighted mean in sRGB. If 2 or fewer groups, the merge is skipped
Refit	recolorize::imposeColors → assignPixels	every pixel to its nearest colour in **Lab**; each colour moved to the Lab mean of its pixels, back to sRGB with clipping
Name	color2label, parse_lut	colour to HSV (rgb2hsv); a CLUT row matches when h, s and v each fall in one of its lo::hi ranges (lo <= x < hi.9999); first matching name wins; no match gives the first row's name
Sum	aggregate	pixel shares summed per name; k = number of names

Where the time goes: `assignPixels` measures every pixel against every colour in an interpreted `sapply` loop, and every image runs alone. Both are what a GPU removes.

3. The GPU design

Many images are processed as one batch. All their foreground pixels go into a single [P, 3] tensor, with a parallel vector giving each pixel's image index. Each step is then one tensor operation over the whole batch:

Step	How it runs
Bin	bin index per pixel = ceil(4x) - 1 per channel; key = image × 64 + bin; bincount and index_add give every image's 64 counts and colour sums at once (GPU)
Merge	at most 64 colours per image, so this runs on the CPU. Complete linkage is written out exactly: merge the closest pair, set the new cluster's distances to the maximum of its parts, stop when the closest pair is above the cutoff. Complete-linkage heights never decrease, so stopping there gives the same partition as cutting the full tree at h = 20. Groups are numbered by first appearance, as cutree numbers them. No scipy dependency
Refit	merged colours padded into an [images, K, 3] tensor with invalid slots masked to infinity; each pixel's distances to its own image's colours in one batched operation, chunked to 2M pixels; argmin; index_add for the Lab means (GPU)
Name	the CLUT is parsed once into ranges; each name's test is a vectorised OR-of-ANDs over all colours or pixels at once; first match wins via a running 'unassigned' mask (GPU)

Colour conversions copy R's grDevices exactly. The sRGB→XYZ matrix is built the way `make.rgb` builds it, from the sRGB primaries and the D65 white point (x 0.3127, y 0.3290). The inverse uses the same Lab constants (ε = 216/24389, κ = 24389/27), and sRGB output is clipped to [0, 1] as with `convertColor(clip = TRUE)`.

R behaviour kept on purpose: the alpha >= 128 cut; right-closed bins; expand.grid bin order; skipping the merge at 2 or fewer groups; nearest-colour search in Lab rather than sRGB; colours with zero pixels after refitting still count toward k; an unmatched colour takes the CLUT's first name.

4. Checking it against R

`parity/parity.R` runs R charisma on a list of images. `parity/parity.py` runs charisma-gpu on the same images and compares colour classes, k and every proportion.

The one bug found was a precision bug. The first comparison, on 40 real cut-outs, matched on 39. The 40th differed by 0.147. Both versions produced the same 18 bins and the same 9 merged groups. Float32 rounding in the pixel reassignment then moved 12 of 62,531 pixels, which shifted one colour from (193.77, 185.19, 13.24) in R to (193.81, 185.21, 13.21). That colour's HSV value is 75.99 in R and 76.01 in Python, and the default CLUT's yellow begins at value 76. So R named the whole 15% cluster green, and Python named it yellow. Moving the pipeline to float64, as R computes, removed the difference: the largest remaining gap is 0.0005.

The same example shows a property of charisma itself: because names go to cluster *averages*, a nudge of 0.02% in one colour can rename 15% of an image.

Test	Images	Same classes	Same k	Max |Δ proportion|
Real cut-outs (*Chrysopsis*), default CLUT	40	40/40	40/40	0.0005
Synthetic (tests/), default CLUT	16	16/16	16/16	< 0.002
Synthetic (tests/), *R. hirta* CLUT	16	16/16	16/16	< 0.002

The synthetic images are made for this purpose: blocks of colour inside and between the CLUT classes, with noise, a brightness gradient, irregular alpha edges, and pixels at alpha 127 to test the cut. Their R results are committed, so `pytest` repeats the comparison without R. Other tests check the Lab values of white and sRGB red, a Lab round trip, HSV against Python's colorsys, the linkage, and the first-row fallback.

5. Per-pixel mode

charisma gives a name to each *merged cluster*, so every pixel in a cluster shares one name. If a cluster straddles two CLUT regions, its average decides for all of it. charisma-gpu also offers `classify_pixels`,

which names every pixel on its own with the same CLUT. On a test set of *R. hirta* flowers, the two modes disagreed most where it matters:

Photo	Bullseye: charisma mode	Bullseye: per pixel	Why
131410147	0.0%	6.0%	maroon patches merged into the disk's cluster
371323113	0.0%	7.4%	near-black ray bases merged into the disk's cluster
256724243	18.9%	9.0%	a disk cluster's average fell in the bullseye region
50311663	49.1%	47.0%	a clear red ring; the two agree

For "how much of this colour is there", the per-pixel share is the more stable number. Both are written out.

6. The *Rudbeckia hirta* CLUT

The goal is to tell three parts of the flower head apart: the yellow **rays**, the red-to-maroon **bullseye** at the ray bases, and the dark **disk**. charisma's default CLUT can't. It calls maroon "brown" and very dark pixels "black", which describes both the bullseye and the disk.

Data. 150 research-grade iNaturalist photos of taxon 62741, spread evenly across the taxon (shard 7 of 480 in photo_id order), were segmented with SAM3 (prompt "flower", score >= 0.5); 135 had at least one mask. Their top segments were inspected by eye. Ten had a clear bullseye, ranging from bright red rings to near-black maroon ray bases. About 22 were plain yellow-rayed flowers.

Step 1: what does the bullseye add? The pixel colours of the plain flowers, whose red and brown pixels can only be disk, were plotted against those of the bullseye flowers (disk plus bullseye). The bullseye adds three clusters the plain flowers lack: bright red (hue –10 to +10°, value 45–95%), red-orange or chestnut (hue 5–26°, saturation >= 75%, value >= 45%), and dark maroon (hue around 345–360°, value 12–45%, saturation >= 70%).

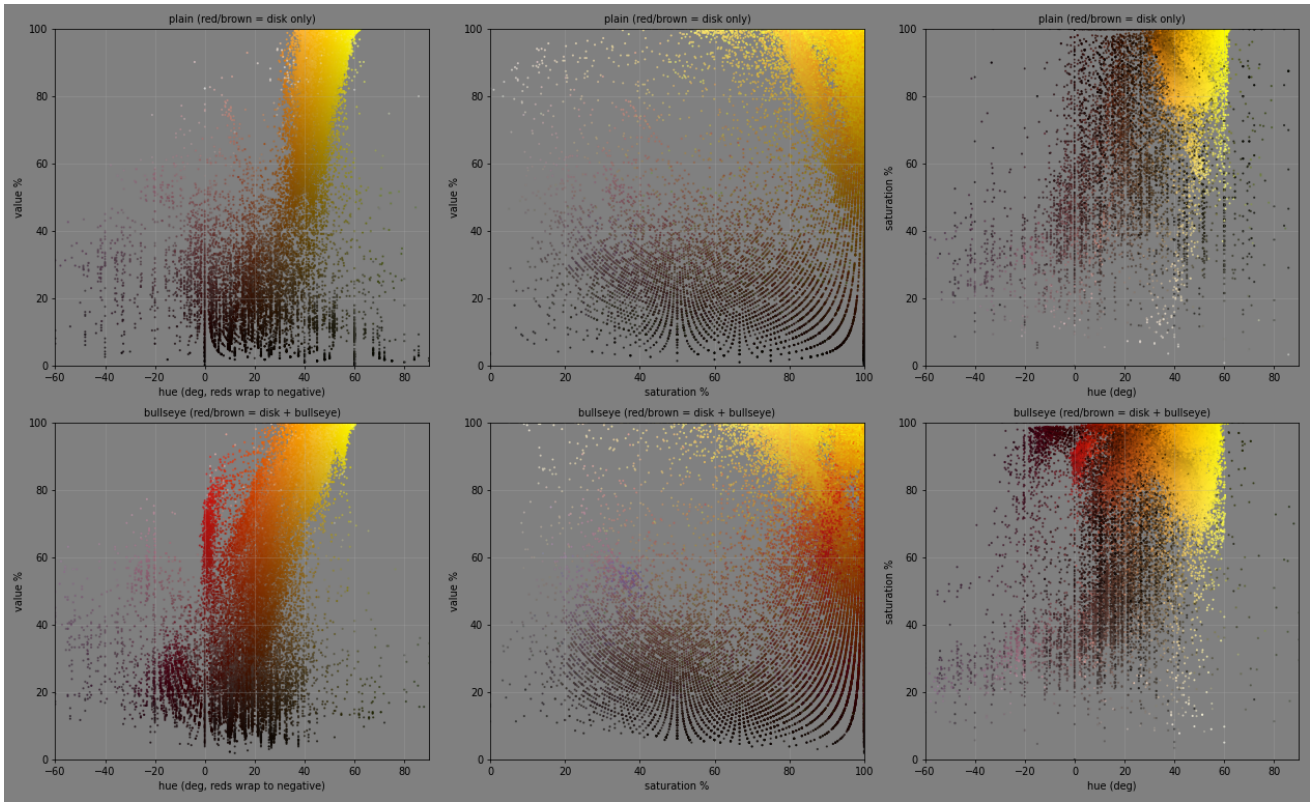


Figure 1. Pixel colours of plain (top) and bullseye (bottom) R. hirta segments, each dot in its own colour. Hue is shown from –60° so reds are continuous.

Step 2: hand-picked pixels. Boxes were drawn on six bullseye flowers: on the bullseye patches and on the disk of the same flower, each checked by eye on an overlay. Two things separate them. Bullseye is **saturated** (10th percentile 57–84%, against disk medians of 31–59%) and **red** (median hue 5°, against 15° for reddish-brown disks; Lab chroma 27 against 17.5).

Step 3: thresholds by search. Candidate rules were scored on three numbers:

- recall: the share of boxed bullseye pixels caught
- the share of boxed disk pixels wrongly caught
- the share of all pixels on the 22 plain flowers called bullseye

A single hue/saturation rule caught 89% of bullseye pixels but put 3.7% bullseye on plain flowers (worst 11%), all of it on reddish-brown disks. Splitting the rule by brightness fixed most of that:

Dark rule (v 10–44)	Bright rule (v >= 45)	Recall	Disk boxes wrong	Plain flowers: mean / worst
h ≤ 9°, s ≥ 65	h ≤ 26°, s ≥ 60	0.742	0.1%	0.7% / 2.8%
h ≤ 12°, s ≥ 65	**h ≤ 26°, s ≥ 60**	**0.839**	**0.3%**	**1.2% / 4.3%**
h ≤ 15°, s ≥ 65	h ≤ 26°, s ≥ 60	0.865	0.5%	1.8% / 6.2%
single rule, s ≥ 65, h ≤ 22°	(same)	0.885	2.6%	3.7% / 11.0%

(Hue ranges also include 330–360°, the red side of 0°.) The bold row is the CLUT:

name	h	s	v	captures
bullseye	0–12 | 330–360; 0–26 | 330–360	65–100; 60–100	10–44; 45–100	maroon (dark) and red to chestnut (bright)
green	71–170	25–100	20–100	leaf or bract inside the mask
disk	0–360; 0–26 | 280–360	0–100; 15–74	0–44; 45–79	everything dark; pinkish-grey disk tops
ray	27–70	25–100	45–100	lemon to golden yellow
pale	0–360	0–24	45–100	white or grey highlights, dried heads
other	0–360	0–100	0–100	the rest

Rows are tried in order, so saturated dark red is claimed by bullseye before the disk's catch-all "anything dark" row sees it. The file uses charisma's CSV format and works unchanged in R charisma, where its results also match (tested).

Limits. The recall and error rates above come from the flowers the thresholds were tuned on, so they're optimistic. On 12 plain flowers not used for tuning, bullseye was 0–2%, except one at 4% that does have a faint red ring. Near-black maroon ray bases are the same colour as dark disks, and no colour rule can separate them. Telling them apart needs position: the disk is the central compact region, and the bullseye lies on the rays. Section 7 adds that step.

7. Red on the petals only

On the full run, the CLUT painted many disks red: reddish-brown and mahogany disks, and disks whose outer florets are bright red-brown, have the colour of a bullseye. Only red on the petals is wanted. `charisma_gpu/petal.py` finds the disk by its shape, after the per-pixel CLUT, on each cut-out's 2-D image:

Step	What it does
Core	foreground pixels that are dark (value < 45) or named disk or other, and not ray, green or pale; holes filled (pink anther centres, highlights)
Disk	the largest connected piece of the core, morphologically opened with a disc of radius 0.6 × its largest inscribed radius r0. Petal-base patches are narrower than the disk and are erased; the round disk survives. Kept only if compact (area <= $4\pi r0^2$), so a ring of maroon petal bases, in a cut-out where SAM3 left the disk out, is not taken for a disk. Grown by 0.15 r0
Red-rimmed disks	the disk is found again with red pixels also seeding the core. That version is kept if its radius is at most 0.28 of the flower's (95th-percentile distance from its centre). A red rim stops at the disk edge (0.15–0.21 of the flower in testing). A bright bullseye ring touching the disk runs out along the petals (0.34–0.50), so it isn't absorbed
Split	bullseye-coloured pixels inside the disk → **disk_red**; elsewhere → **petal_bullseye**

Colour was checked first and doesn't work. Red disk rims and orange-red bullseyes share hue 15–25°, saturation 65–90% and value 45–65%. Texture (dark pixels next to red ones) overlaps too: 0.07–0.36 for rims against 0.03–0.23 for bullseyes.

	Colour only (px_bullseye)	Petals only (petal_bullseye)
22 plain flowers: red share, mean / worst	1.2% / 4.3%	0.13% / 2.2%
Hand-boxed bullseye pixels kept	84% (named bullseye)	74% of those, kept on the petals
... per flower		85–100% on 5 of 6; 41% on 381793369
Disk-box pixels called bullseye	0.3%	0%
Clear rings (50311663, 498837000, 529395248)	47%, 20%, 14%	45%, 20%, 13%

The loss on 381793369 is the case the method can't handle: a maroon ring touching its disk with no gap is one solid blob, and is counted as disk. A disk that is never found (fragmented, or not compact) leaves its red on the petals: in a sample of 30 segments with at least 5% red, that was one fully (68033493, 7%) and one side-view cone partly (386207557, 5%). Orange-rayed flowers can pick up a few percent from red specks on the rays.

8. The full *R. hirta* run

All 126,653 SAM3 segments of taxon 62741's flower-detected photos (38,273 photos, cut-out PNGs, median 1,700 pixels) were classified on the RTX 4060 with no errors. That took about 2 minutes without the petal step, and 3.6 minutes (592 segments/s) with it. Per segment, the median per-pixel shares were 86% ray, 12% disk and 0.2% red.

Segments with red of at least	Colour only	On the petals
5%	10,655	5,515
15%	3,694	2,468
Mean share of the segment	1.9%	0.95% (another 0.98% was on the disk)

The segments with the most red on the petals are red, orange-red and mahogany-rayed forms, where the whole ray is red. A few may be other species. Segments include SAM3's low-scoring masks: filter by mask score once the run's metadata (results.csv) is joined on photo_id and mask_index.

9. Reproducing this

```
pip install -e .
pytest tests/                                  # parity with R, no R needed
charisma-gpu CUTOUTS/ --out colours.csv --clut rudbeckia_hirta
Rscript parity/parity.R paths.txt r.csv [clut.csv]   # your own parity check
python  parity/parity.py paths.txt r.csv [--clut clut.csv]
```

The method is charisma's (Schwartz et al. 2025, doi:10.32614/CRAN.package.charisma) and recolorize's (Weller). Please cite them. Generated by docs/make_methodology_pdf.py.